数据结构与算法综合实训

# 实验二 动态规划实验报告

学号　222024321062099

姓名　谈世钊

班级　软工中外 4 班

## 实验内容与目标

本实验使用 C++17 实现矩阵连乘、最长公共子序列和 0-1 背包，并补充设备更新问题。每个问题均给出状态定义、递推关系、边界条件、计算顺序及最优方案恢复方法。通过典型算例、随机穷举对照和边界测试，验证最优值与输出方案的一致性。

| 实验项目 | 求解目标 | 典型算例结果 |
| --- | --- | --- |
| 矩阵连乘 | 最少标量乘法次数 | 15125 |
| 最长公共子序列 | 最大公共子序列长度 | 4，输出 BCBA |
| 0-1 背包 | 容量约束下的最大价值 | 15，选择 1、2、5 |
| 设备更新 | 规划期内最低总成本 | 255，每台使用 2 年 |

## 实验环境与完成情况

硬件环境为本机 PC，操作系统为 Windows。编译器为 MinGW g++ 13.2.0，采用 C++17 标准，启用 -O2、-Wall、-Wextra 和 -pedantic。程序编译通过，四类算法共完成 800 组随机穷举对照、8 项内置边界检查及 11 项额外命令行测试，全部通过。

报告中的运行图为真实程序输出日志的查看页截图；对应的原始输入、输出、源码和复现脚本保存在实验附件中。

# 一 动态规划原理与总体设计

## 1 最优子结构与重叠子问题

动态规划将原问题组织为相互依赖的子问题，保存已经求出的状态值，并按依赖关系计算新状态。最优子结构要求最优解中包含的子问题解也是相应条件下的最优解；重叠子问题说明不同求解路径会重复访问同一状态，保存结果可以减少重复计算。

最优子结构可以用替换法说明：若最优方案包含一个非最优的子方案，且用更好的子方案替换后不会破坏约束，则原方案还能改进，与最优性矛盾。使用该论证时必须明确状态保留了哪些约束，不能只凭“问题可分解”就使用动态规划。

## 2 状态设计与计算顺序

| 问题 | 状态含义 | 依赖方向 |
| --- | --- | --- |
| 矩阵连乘 | 区间内矩阵乘积的最低代价 | 短区间到长区间 |
| 最长公共子序列 | 两个前缀的 LCS 长度 | 短前缀到长前缀 |
| 0-1 背包 | 前若干件物品在给定容量内的最大价值 | 上一物品层到下一层 |
| 设备更新 | 恰好覆盖若干年的最低成本 | 短规划期到长规划期 |

## 3 从最优值恢复最优方案

仅保存最优值不足以展示完整答案。矩阵连乘记录最优断开点；LCS 按状态相等或相邻最优值回退；背包比较相邻物品层决定是否选择；设备更新记录最后一个使用周期。报告分别检查括号结构、公共子序列合法性、物品总重量和更新周期覆盖长度。

## 4 实验实施步骤

先用小规模数据人工计算期望结果，再实现递推与回溯，最后用不带记忆化的穷举算法作为独立参照。所有随机测试采用固定种子 20261027，便于重现。主程序按 matrix、lcs、knapsack、equipment 和 verify 五种模式运行，各模式的输入格式独立。

本实验采用自底向上的表格法。状态初值、有效范围与循环顺序都在计算前确定；遇到无效维度、非正重量或非法设备费用时，程序输出错误信息并返回非零退出码。

# 二 矩阵连乘问题建模

## 1 问题与输入数据

设有 n 个相容矩阵，矩阵 Ai 的行数为 p(i−1)，列数为 p(i)。矩阵乘法满足结合律，不同括号化方式得到相同的数学乘积，但所需的标量乘法次数可能差别很大。目标是保持矩阵顺序不变，寻找乘法次数最少的括号化方式。

| 矩阵 | 行数 | 列数 |
| --- | --- | --- |
| A1 | 30 | 35 |
| A2 | 35 | 15 |
| A3 | 15 | 5 |
| A4 | 5 | 10 |
| A5 | 10 | 20 |
| A6 | 20 | 25 |

## 2 状态与递推关系

数学说明使用从 1 开始的矩阵编号。M(i,j) 表示 Ai 到 Aj 连乘的最少乘法次数，S(i,j) 保存最后一次乘法的断开位置。单个矩阵无需相乘，故 M(i,i)=0。对于 i<j，枚举最后断开位置 k，左右乘积的维度分别为 p(i−1)×p(k) 和 p(k)×p(j)。

$$M(i,j) = min\{ M(i,k) + M(k+1,j) + p(i-1)\cdot p(k)\cdot p(j) \}$$

其中 k 从 i 到 j−1。每个候选值由左区间成本、右区间成本和最后合并成本组成。选取最小值并保存 k，之后可以递归输出完整括号表达式。

## 3 正确性与数据结构

最优括号结构一定有一个最终断开点。固定该断开点后，如果任一子区间存在更优括号结构，替换后会降低总成本，因此左右子区间必须分别最优。枚举所有最终断开点即可覆盖所有可能最优结构。

程序使用 vector<int> 保存维度，使用二维 vector<long long> 保存代价，使用二维 vector<int> 保存断开点。代码从 0 开始编号，因此合并成本写为 p[i]×p[k+1]×p[j+1]。维度乘法通过 1LL 提升为 64 位计算。

# 三 矩阵连乘算法与关键实现

## 1 伪代码

```
M[i,i] ← 0
for len ← 2 to n:
    for i ← 1 to n-len+1:
        j ← i+len-1; M[i,j] ← INF
        for k ← i to j-1:
            q ← M[i,k]+M[k+1,j]+p[i-1]*p[k]*p[j]
            if q < M[i,j]: M[i,j] ← q; S[i,j] ← k
PrintOrder(i,j):
    if i=j: output Ai
    else: output '(' + PrintOrder(i,S[i,j])
                 + '×' + PrintOrder(S[i,j]+1,j) + ')'
```

## 2 关键 C++ 代码

```
// i、j、k 为 0 基下标；按区间长度递增计算。
for (int len = 2; len <= n; ++len)
    for (int i = 0; i + len <= n; ++i) {
        int j = i + len - 1;
        r.cost[i][j] = INF;
        for (int k = i; k < j; ++k) {
            ll q = r.cost[i][k] + r.cost[k + 1][j]
                 + 1LL * p[i] * p[k + 1] * p[j + 1];
            if (q < r.cost[i][j]) {
                r.cost[i][j] = q;
                r.split[i][j] = k; // 保存回溯决策
            }
        }
    }
```

## 3 实现要点

如果直接按照 i、j 的普通双重循环填表，可能在子区间尚未计算时就读取其初值。按 len 从小到大计算能够确保依赖全部就绪。严格小于时才更新，使相同成本的多个答案具有稳定的断开点选择。

代价表只有上三角有效；截图中的下三角 0 是未使用的存储单元。INF 取 long long 最大值的四分之一。输入最多 100 个矩阵、每个维度不超过 $10^5$，最多 99 次矩阵合并的总代价不超过 $9.9\times10^{16}$，小于 INF，也小于 64 位有符号整数上界。

# 四 矩阵连乘运行结果与核验




**图 1 矩阵连乘典型算例的实际输入和输出**

## 1 结果解释

程序得到最少乘法次数 15125，括号结构为 ((A1×(A2×A3))×((A4×A5)×A6))。维度数组为 30、35、15、5、10、20、25。

| 计算阶段 | 标量乘法次数 | 阶段结果维度 |
|---|---|---|
| A2×A3 | 2625 | 35×5 |
| A1×(A2×A3) | 5250 | 30×5 |
| A4×A5 | 1000 | 5×20 |
| (A4×A5)×A6 | 2500 | 5×25 |
| 左右两部分合并 | 3750 | 30×25 |

上表成本之和为 15125。按原顺序从左向右相乘的成本为 40500，优化后减少 25375 次，约为 62.65%。该比较仅表示此算例的运算次数差异，不等同于所有输入都能获得相同比例的时间收益。

# 五 最长公共子序列问题建模

## 1 子序列与子串的区别

子序列保留原字符的先后顺序，但不要求连续；子串要求连续。本实验寻找两个字符串的最长公共子序列，不能在字符不相等时直接把状态置零。典型输入 X=ABCBDAB，Y=BDCABA，目标是输出长度和一个具体的公共子序列。

## 2 状态与边界条件

L(i,j) 表示 X 的前 i 个字符和 Y 的前 j 个字符的 LCS 长度。任意字符串与空串的公共子序列长度都为 0，因此第 0 行和第 0 列初始化为 0。

$$L(i,j) = L(i-1,j-1) + 1$$

上式适用于 X 的第 i 个字符与 Y 的第 j 个字符相同的情况；相同末尾字符可接在较短前缀的公共子序列后面。若两者不同，至少有一个末尾字符不在当前最优公共子序列中。

$$L(i,j) = max\{L(i-1,j), L(i,j-1)\}$$

## 3 计算与恢复路径

从左上向右下填表，每个状态只依赖上方、左方和左上方。回溯从右下角开始：字符相等则记录该字符并同时减少 i 和 j；否则进入较大的相邻状态；相等时固定向上回退。回溯得到的字符顺序是反向的，最后必须反转。

| 数据结构 | 用途 | 规模 |
| --- | --- | --- |
| string x、y | 保存两个输入序列 | m+n 个字符 |
| 二维整数表 dp | 保存各前缀的 LCS 长度 | (m+1)×(n+1) |
| string sequence | 保存恢复出的一个 LCS | 最多 min(m,n) 个字符 |

## 4 正确性说明

若末尾字符不同，任何公共子序列不可能同时使用这两个末尾字符，因此它一定属于去掉 X 末尾或去掉 Y 末尾的某个子问题。若末尾相同，可在相应较短前缀的最优公共子序列后加入这一字符。结合空串边界，按前缀长度递增的归纳可得到最终最优长度。

LCS 可能不唯一，不能只把某一个固定字符串当成唯一正确答案。验证时同时检查长度最优、结果为 X 的子序列、结果为 Y 的子序列。

# 六 最长公共子序列算法与关键实现

## 1 伪代码

```
L[0,*] ← 0; L[*,0] ← 0
for i ← 1 to m:
    for j ← 1 to n:
        if X[i]=Y[j]: L[i,j] ← L[i-1,j-1]+1
        else: L[i,j] ← max(L[i-1,j],L[i,j-1])
i ← m; j ← n; answer ← empty
while i>0 and j>0:
    if X[i]=Y[j]: append X[i]; i←i-1; j←j-1
    else if L[i-1,j] >= L[i,j-1]: i←i-1
    else: j←j-1
reverse(answer)
```

## 2 关键 C++ 代码

```
for (int i = 1; i <= m; ++i)
    for (int j = 1; j <= n; ++j)
        r.dp[i][j] = x[i - 1] == y[j - 1]
            ? r.dp[i - 1][j - 1] + 1
            : max(r.dp[i - 1][j], r.dp[i][j - 1]);
// 回溯构造一个答案；平局时向上移动。
for (int i = m, j = n; i > 0 && j > 0;) {
    if (x[i - 1] == y[j - 1]) {
        r.sequence += x[i - 1]; --i; --j;
    } else if (r.dp[i - 1][j] >= r.dp[i][j - 1]) --i;
    else --j;
}
reverse(r.sequence.begin(), r.sequence.end());
```

## 3 输入与边界处理

程序使用 getline 分别读取两行，允许某行为空。读取 Windows 换行时去掉行末可能残留的回车字符。字符串长度均限制为 2000，以控制二维表的内存开销。命令行输入模式不先用 >> 读取数字，因而不会遗留一个多余换行。

当前字符串以字节作为序列元素，适合本实验的 ASCII 英文字母。若将该实现用于中文 UTF-8 文本，应先将字符串解码为 Unicode 码点序列，再执行同样的动态规划；不能把 UTF-8 字节序列的公共片段直接解释为中文字符。

# 七 最长公共子序列运行结果与核验




**图 2 最长公共子序列的实际运行记录**

## 1 长度与方案核验

程序输出长度 4，序列 BCBA。在 X=ABCBDAB 中，可以选择第 2、3、4、6 个字符；在 Y=BDCABA 中，可以选择第 1、3、5、6 个字符。两个位置序列都严格递增，因此 BCBA 是合法公共子序列。

## 2 状态表解释

截图首行、首列为 0，对应空前缀。最后一行最后一列为 4，表明完整字符串的最优长度为 4。字符不相等时，状态沿上方或左方传播已有最优值，因此表中出现连续不变区域属于正常现象。

另一种长度为 4 的公共子序列可能同样正确。本实现平局时向上回退，因此稳定输出 BCBA；不能因为输出与其他程序不同就判定错误。随机测试通过枚举 X 的所有子序列求真实最优长度，再分别检查输出在 X 和 Y 中的合法性。

## 3 补充算例

空串与 ABC 的期望长度为 0；ABC 与 XYZ 的期望长度为 0；AAAA 与 AA 的期望长度为 2。这三类输入分别检查空边界、无公共字符和重复字符的处理。

# 八 0-1 背包问题建模

## 1 输入与约束

给定 n 件物品，第 i 件重量为 w(i)，价值为 v(i)，背包容量为 C。每件物品最多选一次，要求总重量不超过 C，且总价值最大。允许不装满背包，也允许空集合；本实验要求重量为正整数、价值为非负整数。

| 物品编号 | 重量 | 价值 |
| --- | --- | --- |
| 1 | 2 | 6 |
| 2 | 2 | 3 |
| 3 | 6 | 5 |
| 4 | 5 | 4 |
| 5 | 4 | 6 |

典型输入的容量为 10。重量相同的第 1、2 件物品仍然是不同物品，状态必须保留处理到哪一件物品的信息，不能按重量去重。

## 2 状态转移

B(i,c) 表示仅使用前 i 件物品、容量不超过 c 时可获得的最大价值。第 0 层均为 0。对于第 i 件物品，若重量超过 c，则只能不选；否则比较不选与选取后的价值。

$$B(i,c) = max\{B(i-1,c), B(i-1,c-w(i)) + v(i)\}$$

上式仅在 c≥w(i) 时使用。选取分支必须读取上一层 i−1 的状态，避免同一物品在同一层被反复选入。边界 B(i,0)=0 来自容量为零且重量为正的约束。

## 3 正确性与方案恢复

最优方案要么不含第 i 件物品，要么包含它。不含时答案属于前 i−1 件的子问题；包含时去掉该物品，余下物品必须在剩余容量内达到最优，否则可替换得到更大价值。两类互斥情况覆盖所有可行方案。

程序保存 (n+1)×(C+1) 个 long long 状态。回溯时若 B(i,c)>B(i−1,c)，则选择第 i 件并减去其重量；否则跳过。相等时跳过属于一种稳定的平局策略，并不影响最优价值。

# 九 0-1 背包算法与关键实现

## 1 伪代码

```
B[0,*] ← 0
for i ← 1 to n:
    for c ← 0 to C:
        B[i,c] ← B[i-1,c]
        if w[i] <= c:
            B[i,c] ← max(B[i,c],B[i-1,c-w[i]]+v[i])
c ← C; answer ← empty
for i ← n downto 1:
    if B[i,c] > B[i-1,c]:
        append i to answer; c ← c-w[i]
reverse(answer)
```

## 2 关键 C++ 代码

```
for (int i = 1; i <= n; ++i)
    for (int c = 0; c <= cap; ++c) {
        r.dp[i][c] = r.dp[i - 1][c]; // 不选第 i 件
        if (w[i - 1] <= c)
            r.dp[i][c] = max(r.dp[i][c],
                r.dp[i - 1][c - w[i - 1]] + v[i - 1]);
    }
int c = cap;
for (int i = n; i >= 1; --i)
    if (r.dp[i][c] > r.dp[i - 1][c]) {
        r.items.push_back(i); // 记录物品编号
        c -= w[i - 1];
    }
reverse(r.items.begin(), r.items.end());
```

## 3 空间优化与适用限制

如果只需要最优价值，可以把二维表压缩为一维数组，但容量必须从 C 向下遍历至当前重量。逆序保证读取的是本轮处理之前的状态；正序会允许当前物品重复使用，变成完全背包的含义。

本实验保留二维表，以清楚展示各阶段结果并可靠恢复选择方案。输入最多 500 件物品、容量最多 10000，每件价值不超过 $10^9$；累计价值和状态值使用 long long。容量较大时，按容量展开的算法可能占用大量内存，需要根据实际参数选择其他实现。

# 十 0-1 背包运行结果与核验




**图 3 0-1 背包典型算例实际运行记录**

## 1 最优值与约束检查

程序输出最大价值 15，选择第 1、2、5 件。总重量为 2+2+4=8，不超过容量 10；总价值为 6+3+6=15。该结果说明最优背包不一定恰好装满，剩余 2 个容量单位无法放入其他未选择物品。

## 2 状态变化分析

处理第 3 件物品后，容量 10 的最优值为 14，对应第 1、2、3 件。处理第 4 件后该位置仍为 14；处理第 5 件后变为 15，最终方案改为第 1、2、5 件。这体现动态规划会综合比较候选方案，而不是只按单件价值或重量作局部选择。

## 3 区分 0-1 与完全背包

额外测试仅有一件重量 2、价值 3 的物品，容量为 4。正确结果仍为 3，而不是 6。该测试已实际运行并通过，可有效发现错误使用当前层状态或一维正序更新导致的重复选择问题。

# 十一 设备更新问题建模

## 1 模型假设与规划目标

设置一个自定义设备更新算例：从第 1 年开始购入新设备，需连续服务 N 年，设备最多使用 L 年。每台新设备购价相同；维护费只与设备年龄有关；设备在年末出售并获得相应残值。规划结束时也出售设备。忽略折现、通货膨胀、故障停机和期初已有设备。目标是最小化整个规划期的净支出。

| 设备年龄 k | 当年维护费 | 年末残值 | 使用 k 年的净成本 |
|---|---|---|---|
| 1 | 10 | 60 | 50 |
| 2 | 20 | 45 | 85 |
| 3 | 40 | 30 | 140 |
| 4 | 70 | 20 | 220 |
| 5 | 110 | 10 | 340 |

典型输入 N=6、L=5、购价 P=100，金额单位统一使用算例成本单位。使用 k 年的周期成本 C(k) 等于购价加第 1 至 k 年维护费之和，再减去第 k 年末残值。维护费是逐年费用，不能只计第 k 年一年的维护费。

## 2 状态与递推

F(t) 表示恰好服务 t 年且在期末出售当前设备的最低总成本。初值 F(0)=0。设最后一台设备使用 k 年，则前面 t−k 年必须已经达到最优。

$$F(t) = min\{F(t-k) + C(k)\}$$

k 从 1 到 min(t,L)。保存使成本最小的 k，即可从 t=N 逐次减去最后一段长度，得到全部更新周期。周期成本使用前缀和预处理，每次转移只需常数时间。

## 3 正确性与适用范围

任意可行更新方案都能表示为若干正整数使用年限之和，每段不超过 L。递推枚举最后一段长度，覆盖全部整数分段；如果前面的 t−k 年不是最优，替换后可降低总支出。因此该模型下的递推得到全局最优。若购价随年份变化或考虑贴现，需要把购入年份和折现因素纳入成本模型。

# 十二 设备更新算法与关键实现

## 1 伪代码

```
sum ← 0
for k ← 1 to L:
    sum ← sum + maintenance[k]
    C[k] ← P + sum − salvage[k]
F[0] ← 0; F[1..N] ← INF
for t ← 1 to N:
    for k ← 1 to min(t,L):
        if F[t−k]+C[k] < F[t]:
            F[t] ← F[t−k]+C[k]; previous[t] ← k
t ← N
while t>0: append previous[t]; t ← t−previous[t]
reverse(answer)
```

## 2 关键 C++ 代码

```
ll sum = 0;
for (int k = 1; k <= life; ++k) {
    sum += maintenance[k - 1];
    r.cycle[k] = price + sum - salvage[k - 1];
}
r.dp[0] = 0;
for (int t = 1; t <= years; ++t)
    for (int k = 1; k <= min(t, life); ++k)
        if (r.dp[t - k] + r.cycle[k] < r.dp[t]) {
            r.dp[t] = r.dp[t - k] + r.cycle[k];
            r.previous[t] = k; // 保存最后一段年限
        }
for (int t = years; t > 0; t -= r.previous[t])
    r.lives.push_back(r.previous[t]);
reverse(r.lives.begin(), r.lives.end());
```

## 3 数据结构与边界处理

cycle 长度为 L+1，dp 和 previous 长度均为 N+1，使用年限保存到 lives 中。购价与维护费不得为负，残值应处于 0 至购价之间。N=0 时最小成本为 0，回溯循环不会执行，因此不会错误地产生购置记录。

每次转移的 k 至少为 1，故回溯的剩余年份严格减少。实际主程序还限制 $N \le 10000$、$L \le 100$、购价及年度维护费 $\le 10^9$，使计算和存储成本保持可控。

# 十三 设备更新运行结果与核验

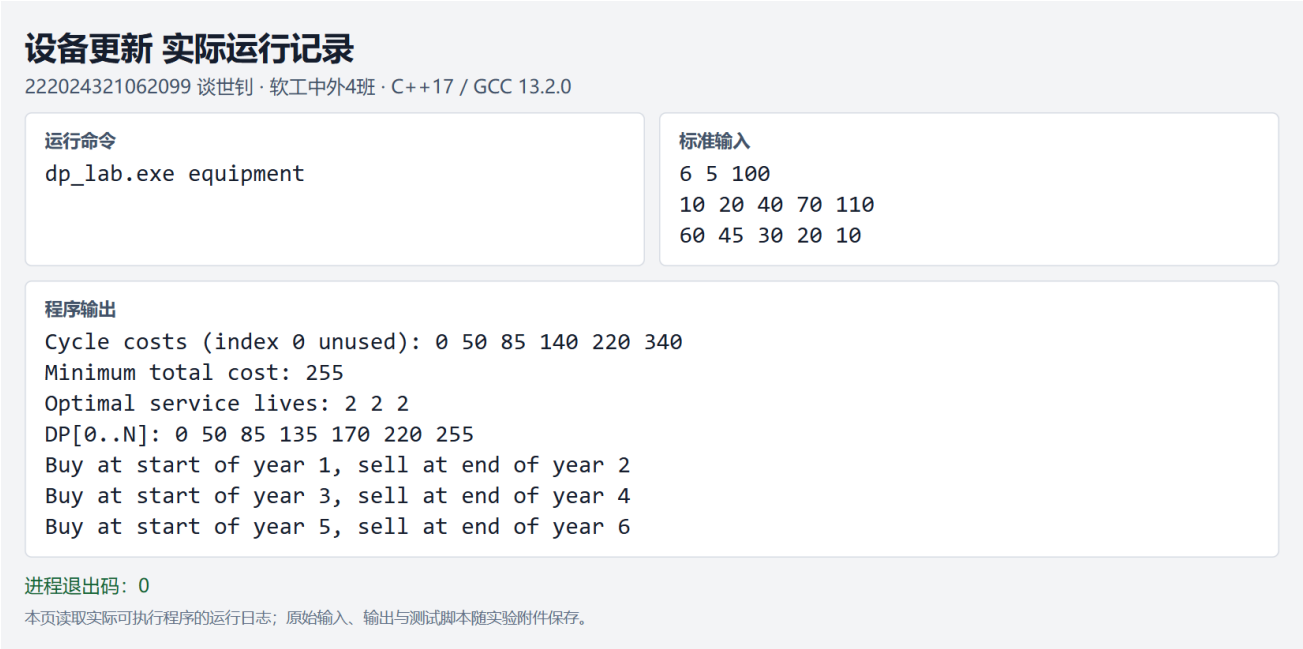



**图 4 设备更新问题实际运行记录**

## 1 最优更新计划

程序得到最低总成本 255，三台设备分别使用 2 年。第 1、3、5 年初购置设备，第 2、4、6 年末出售设备。服务区间连续覆盖 6 年，每台使用年限均不超过上限 5 年。

| 使用周期 | 购价 | 维护费合计 | 出售残值 | 净成本 |
|---|---|---|---|---|
| 第 1 至 2 年 | 100 | 30 | 45 | 85 |
| 第 3 至 4 年 | 100 | 30 | 45 | 85 |
| 第 5 至 6 年 | 100 | 30 | 45 | 85 |
| 合计 | 300 | 90 | 135 | 255 |

## 2 状态与备选方案分析

各年最优成本为 0、50、85、135、170、220、255。对于 6 年，最后一段分别使用 1 至 5 年时，候选总成本依次为 270、255、275、305、390，因此选择最后使用 2 年，并继续回溯得到 2、2、2。

每年更新的成本为 6×50=300，两个 3 年周期的成本为 2×140=280，均高于 255。保持一台设备服务 6 年超过寿命上限，不属于可行备选方案。

# 十四 随机穷举对照测试




**图 5 固定随机种子的穷举对照与内置边界测试**

## 1 对照方法与覆盖范围

| 算法 | 每类组数 | 穷举参照与规模 |
|---|---|---|
| 矩阵连乘 | 200 | 1 至 7 个矩阵；递归枚举全部括号化 |
| 最长公共子序列 | 200 | 串长 0 至 8；枚举第一个串的全部子序列 |
| 0-1 背包 | 200 | 0 至 10 件；枚举全部物品子集 |
| 设备更新 | 200 | 0 至 8 年；枚举全部合法整数分段 |

随机数发生器采用 C++ mt19937，固定种子为 20261027。穷举参照直接按原始可行方案搜索，不使用动态规划表，也不复用待验证算法的最优值。四类各 200 组，随机对照共 800 组。

## 2 同时验证数值与方案

LCS 除了对比最优长度，还检查输出是否分别为两个输入串的子序列。背包重新累计回溯方案的重量与价值，并检查容量约束。设备更新重新累计每段费用和使用年限。矩阵连乘对照枚举全部括号化的最小代价，典型算例另行手算核验括号结构。

800 组随机对照与 8 项内置检查均通过。该结果提高了对实现正确性的信心，但有限样本不能代替对所有输入的数学证明，特别是超出已声明输入范围的情况。

# 十五 边界与非法输入测试




**图 6 额外命令行边界与非法输入检查**

## 1 边界用例的作用

单矩阵检验初值和零次运算；空串检验第 0 行或第 0 列；重复字符检验回溯；容量为零检验可行空集合；单件物品与两倍容量检验是否发生重复取用；零规划期检验回溯停止；寿命为 1 检验连续逐年更新。

## 2 数值范围与错误退出

两个 50000×50000 的矩阵相乘需要 125000000000000 次标量乘法，明显超过 32 位有符号整数范围。该用例实际输出正确，说明维度乘积没有先在 int 范围内溢出后再赋给 long long。

零维度矩阵、零重量物品和负维护费均被拒绝。测试脚本同时检查错误信息包含 ERROR、退出码为 1，避免仅打印提示却继续输出看似有效的结果。额外命令行测试共 11 项，全部通过；与前一页的 808 项检查合计 819 项，其中部分边界场景有意重叠。

# 十六 时间与空间复杂度分析

## 1 矩阵连乘

长度为 len 的区间共有 n−len+1 个，每个区间枚举 len−1 个断开点。所有候选转移数等于对 len=2 至 n 的 (n−len+1)(len−1) 求和，结果为 $(n^3-n)/6$，因此填表时间为 $O(n^3)$。两张 n×n 表的空间为 $O(n^2)$。括号恢复访问各子区间结构；递归深度最坏为 O(n)。

## 2 最长公共子序列

共有 (m+1)(n+1) 个状态，每个非边界状态只做常数次比较和赋值，所以填表时间为 O(mn)。回溯每一步至少减小一个下标，最多 m+n 步，时间 O(m+n)。二维表空间 O(mn)，输出序列最多 min(m,n) 个字符。若只求长度，可以滚动两行并将较短串安排在列方向，把空间降至 O(min(m,n))。

## 3 0-1 背包

每件物品遍历从 0 到 C 的容量，共 n(C+1) 次，因此填表时间为 O(nC)。回溯逐件检查，时间为 O(n)。二维表占 O(nC) 空间；只求价值时一维数组可降为 O(C)。这里的 O(nC) 是伪多项式复杂度，因为输入整数 C 的二进制表示长度为 O(log C)，容量数值很大时并不意味着算法仍然高效。

## 4 设备更新

周期成本预处理为 O(L)。每个年份 t 枚举 min(t,L) 个周期，转移总数不超过 NL，所以总时间为 O(NL+L)，通常记为 O(NL)。dp、previous 与 cycle 的总空间为 O(N+L)。回溯每段至少覆盖一年，最多 N 段，时间 O(N)。

| 算法 | 填表时间 | 主要存储 | 恢复答案 |
| --- | --- | --- | --- |
| 矩阵连乘 | $O(n^3)$ | $O(n^2)$ | 保存断开点 |
| LCS | O(mn) | O(mn) | 沿表回溯 |
| 0-1 背包 | O(nC) | O(nC) | 逐件回溯 |
| 设备更新 | O(NL) | O(N+L) | 逐段回溯 |

## 5 资源估计与取舍

典型算例均很小，主要用于观察状态与核验答案，而非测量性能。LCS 在两串长度均为 2000 时，整数表约占 15.3 MiB；背包 n=500、C=10000 时，64 位表约占 38.2 MiB，不含容器等附加开销。保留完整表可使回溯和教学观察更直观，处理更大规模时应重新评估。

# 十七 复现步骤与实现问题分析

## 1 文件与编译运行

完整源文件位于 src/dp_lab.cpp。evidence 目录保存每项实验的 input.txt、output.txt、截图和 records.json。run_experiments.py 调用实际可执行程序并进行额外断言；capture_results.cjs 读取日志并保存查看页截图。

```
g++ -std=c++17 -O2 -Wall -Wextra -pedantic `
    src/dp_lab.cpp -o dp_lab.exe
dp_lab.exe matrix
dp_lab.exe lcs
dp_lab.exe knapsack
dp_lab.exe equipment
dp_lab.exe verify
```

前四个模式运行后，分别输入相应截图中的标准输入内容；verify 不需手工输入。matrix 首行是矩阵数量，随后是 n+1 个维度；lcs 直接输入两行字符串；knapsack 输入物品数和容量，再输入每件重量与价值；equipment 输入年份数、最大寿命、购价，再输入维护费数组和残值数组。

## 2 关键问题及处理方法

下标映射：数学推导采用 1 基编号，C++ 容器使用 0 基下标。矩阵维度映射、LCS 字符访问和背包物品编号均明确转换。输出物品及矩阵编号从 1 开始，便于与题意对应。

最优值与方案一致性：如果只检查最优数值，回溯仍可能有下标或反序错误。实验对输出方案重新计算约束和代价，LCS 与设备周期在回溯后均反转，避免把逆向搜索顺序直接作为最终答案。

模型边界：背包按“不超过容量”建模，允许剩余容量；设备更新包含期末残值，避免漏扣最后一台设备残值；矩阵连乘中的乘法次数使用 64 位整数，不把二维矩阵数据本身读入内存。

错误处理：输入非法时通过异常返回错误信息和退出码 1。非正重量、无效维度和非法维护费已执行专门的负向测试。程序的输入限制是为本实验实现设置的范围，不能据此推断一般问题只能处理这些规模。

# 十八 实验总结与改进建议

## 1 实验结果与目标完成情况

本实验完成四类动态规划程序，典型算例的最优值分别为 15125、4、15 和 255。每个问题均从输入约束出发定义状态，给出递推、边界、求值顺序和方案恢复方法；运行结果与人工核验及小规模穷举参照一致。源代码采用标准容器并对关键步骤添加注释，便于按模块阅读和复现。

## 2 方法上的收获

四个问题的共同点在于把选择的影响保留在状态中，然后枚举所有合法决策。它们的差别主要体现在依赖结构：矩阵连乘按区间长度组织，LCS 按两个前缀组织，背包增加物品阶段，设备更新把时间划分为使用周期。因此，设计循环前应先画清状态依赖，而不是直接套用双重或三重循环。

正确性需要同时考虑最优性与可行性。一个数字即使与样例相同，也不能说明输出方案满足原约束。LCS 的字符顺序、背包的取用次数和设备更新的连续覆盖都需要单独验证。将这些约束写成断言，可以让测试发现只看最后一个状态值时容易遗漏的问题。

## 3 得失与局限

完整二维表让状态变化和回溯过程清晰，但也带来较大的内存占用。随机穷举对照覆盖了大量小规模输入，便于发现边界和转移错误，但不能衡量超大输入下的速度，也不能保证未建模现实因素的合理性。设备更新算例采用固定购价和无折现假设，其答案只在该假设下成立。

本次没有把运行耗时作为结论依据，避免对非常短的程序用单次测量得出性能排序。后续若开展效率实验，应在相同编译配置下选取多个规模、重复运行，记录统计量，并把算法计算、输入输出和截图生成的时间分开。

## 4 改进方向与实验建议

实现方面，可增加只求最优值的空间压缩模式，并保留现有完整表模式用于解释和回溯；为 LCS 增加码点级输入处理；为设备更新增加随年份变化的价格与折现。验证方面，可增加最大允许规模的资源检查，以及更多精心构造的平局和退化算例。

实验任务说明可进一步明确选做题目数量，并区分列出的选题与设备更新要求。对实验教学，建议先用小表格手算说明状态含义，再要求给出独立对照方法，使报告不仅展示运行截图，还能够解释结果为什么正确。